

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Report

Laboratory work No. 4

Operating Systems

The xv6 Shell from the Outside

FAF-241, Cosneanu Corina

Checked by:
Reitamn Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chișinău – 2026

Contents

1 PURPOSE OF THE WORK **1**
1.1 Tasks 1

2 PART 1. FIXING `sleep` FOR THE CURRENT XV6 **2**
2.1 Checking the system call name 2
2.2 The updated program 2
2.3 Build and test 3

3 PART 2. BASH VERSUS THE XV6 SHELL **5**
3.1 Method 5
3.2 Comparison 6
3.3 The PATH test in xv6 7
3.4 Observe: answers 8

4 PART 3. WHO EXPANDS ∗? A PROGRAM THAT PRINTS ITS ARGV **10**
4.1 The program for xv6 10
4.2 The program for Linux 10
4.3 Results in xv6 11
4.4 Results in bash 11
4.5 Observe: answers 12

5 PART 4. READING THE SHELL **13**
5.1 Commands run 13
5.2 Observe: answers 14

6 CONCLUSIONS **15**

1 PURPOSE OF THE WORK

The whole user interface of xv6 is one program, `user/sh.c`, about 500 lines long. The purpose of this laboratory work is to use the xv6 shell side by side with `bash`, to discover by experiment what a shell must do and what `bash` adds on top, and to read the source code of the xv6 shell.

The work also includes a small program that prints the arguments it receives. It shows exactly which component expands `*` and removes quotes: the program, the kernel or the shell. Before that, the `sleep` program from Lab 2 is fixed so that it builds on the current xv6.

1.1 Tasks

1. Fix `sleep` for the current xv6, where the system call `sleep()` was renamed to `pause()`.
2. Compare `bash` and the xv6 shell on a set of commands.
3. Write the program `args`, which prints `argv`, for xv6 and for Linux, and run it in both shells.
4. Read `user/sh.c` and identify the kinds of commands and the place where `cd` is handled.

2 PART 1. FIXING `sleep` FOR THE CURRENT XV6

2.1 Checking the system call name

In August 2025 the xv6 authors renamed the system call `sleep()` to `pause()`. The name used by the current tree was checked in `user/user.h`:

```
cd ~/xv6-riscv
grep -n "pause\|sleep" user/user.h
```

The result is the line `25:int pause(int);`, so the tree uses the new name `pause`.

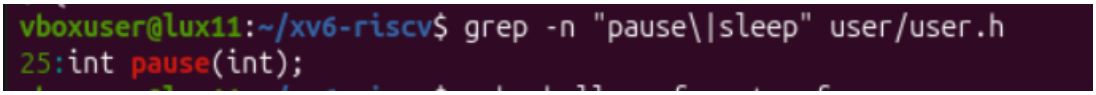


Figure 1.1 – The grep result in user.h

2.2 The updated program

The file `user/sleep.c` calls `pause()` instead of `sleep()`:

```
#include "kernel/types.h"
#include "user/user.h"
int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));  // was sleep() before Aug 2025
  exit(0);
}
```

The program must also be listed in `UPROGS` in the `Makefile`. The line was added after `$U/_zombie`; it starts with a TAB and nothing follows the backslash:

```
	$U/_sleep\
```

```
vboxuser@lux11:~/xv6-riscv$ cat > user/sleep.c << 'EOF'
#include "kernel/types.h"
#include "user/user.h"
int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));
  exit(0);
}
EOF
vboxuser@lux11:~/xv6-riscv$ cat user/sleep.c
#include "kernel/types.h"
#include "user/user.h"
int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));
  exit(0);
}
vboxuser@lux11:~/xv6-riscv$ grep -n "sleep" Makefile
24:  $K/sleeplock.o \
vboxuser@lux11:~/xv6-riscv$ grep -n "UPROGS\|_wc\|_zombie" Makefile
132:UPROGS=\
147:    $U/_wc\
148:    $U/_zombie\
154:fs.img: mkfs/mkfs README $(UPROGS)
```

Figure 1.2 – The updated sleep.c

2.3 Build and test

The system was built and started with `make qemu`. At the xv6 prompt:

```
$ sleep 20
$ sleep
usage: sleep <ticks>
```

With an argument, `sleep` waits for 20 ticks and returns to the prompt without any message. Without an argument it prints the usage message and exits with status 1.

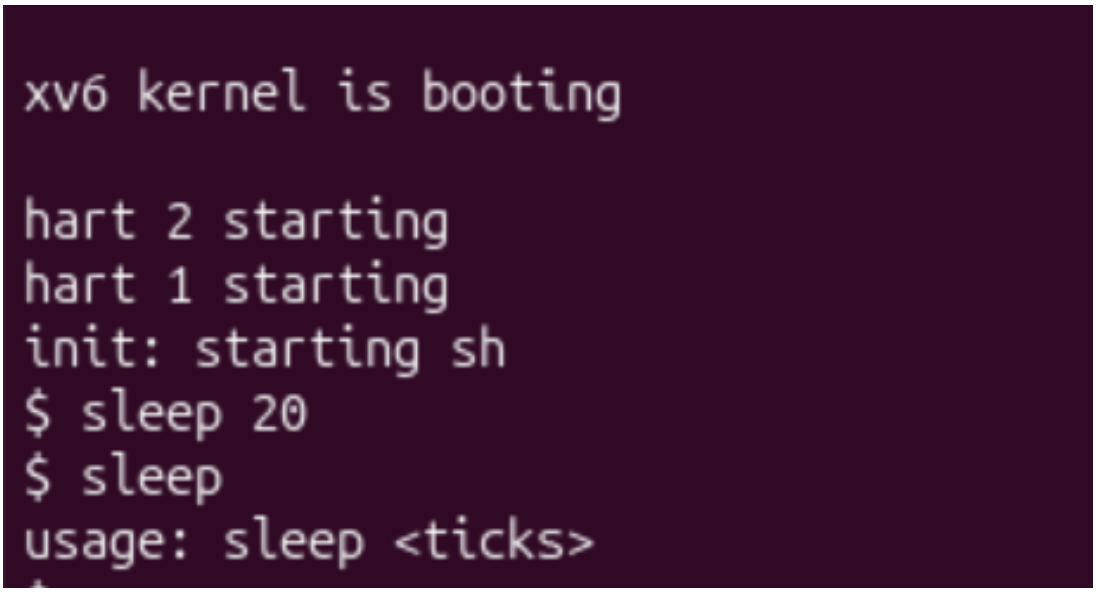


Figure 1.3 – sleep running in xv6, with and without an argument

3 PART 2. BASH VERSUS THE XV6 SHELL

3.1 Method

Every command was run twice: in `bash`, in an empty test folder containing only a file named `README`, and at the xv6 `$` prompt.

```
mkdir ~/shtest && cd ~/shtest && touch README
```

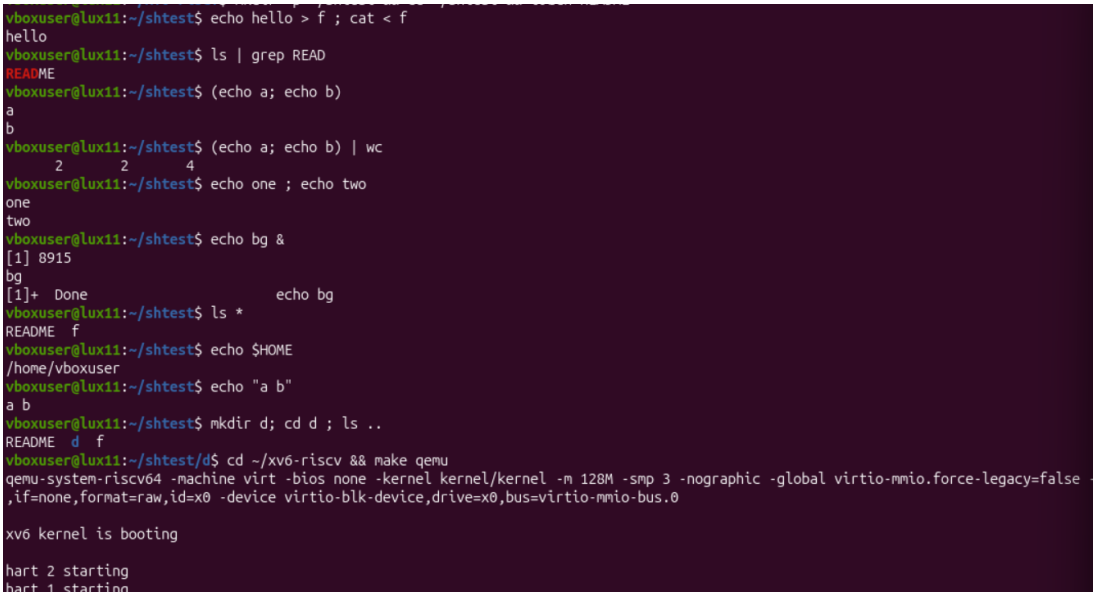


Figure 2.1 – The commands run in bash

```
$ QEMU: Terminated
vboxuser@lux11:~/xv6-riscv$ grep -n "pause\|sleep" user/user.h
25:int pause(int);
vboxuser@lux11:~/xv6-riscv$ echo hello > f; cat < f
hello
vboxuser@lux11:~/xv6-riscv$ ls | grep READ
README
vboxuser@lux11:~/xv6-riscv$ (echo a; echo b) | wc
      2       2       4
vboxuser@lux11:~/xv6-riscv$ echo one; echo two
one
two
vboxuser@lux11:~/xv6-riscv$ echo bg &
[1] 8770
bg
[1]+  Done                    echo bg
vboxuser@lux11:~/xv6-riscv$ ls *
LICENSE  Makefile  README  f  fs.img  test-xv6.py

kernel:
bio.c      file.h       log.o        proc.h       string.o     trap.o
bio.d      file.o       main.c       proc.o       swtch.S      types.h
bio.o      fs.c         main.d       riscv.h      swtch.o      uart.c
buf.h      fs.d         main.o       sleeplock.c  syscall.c    uart.d
console.c  fs.h         memlayout.h  sleeplock.d  syscall.d    uart.o
console.d  fs.o         param.h      sleeplock.h  syscall.h    virtio.h
console.o  kalloc.c     pipe.c       sleeplock.o  syscall.o    virtio_disk.c
defs.h     kalloc.d     pipe.d       spinlock.c   sysfile.c    virtio_disk.d
elf.h      kalloc.o     pipe.o       spinlock.d   sysfile.d    virtio_disk.o
entry.S    kernel       plic.c       spinlock.h   sysfile.o    vm.c
entry.o    kernel.asm   plic.d       spinlock.o   sysproc.c    vm.d
exec.c     kernel.ld    plic.o       start.c      sysproc.d    vm.h
exec.d     kernel.sym   printk.c     start.d      sysproc.o    vm.o
exec.o     kernelvec.S  printk.d     start.o      trampoline.S
fcntl.h    kernelvec.o  printk.o     stat.h       trampoline.o
file.c     log.c        proc.c       string.c     trap.c
file.d     log.d        proc.d       string.d     trap.d
```

Figure 2.2 – The same commands run in xv6

3.2 Comparison

Redirection and sequence (`echo hello > f ; cat < f`). Both shells print `hello`. They handle `>`, `<` and `;`. In xv6 this is the REDIR and LIST command type.

Pipe (`ls | grep READ`). The pipe works in both shells. `bash` prints `README`, while xv6 prints `README 2 2 2441`, because the xv6 `ls` shows the name, type, inode and size, not only the name.

Grouping (`(echo a; echo b) | wc`). Both shells print `2 2 4`: 2 lines, 2 words and 4 characters. Grouping with parentheses and the pipe work the same way.

Sequence (`echo one ; echo two`). Both shells print `one` and `two`. The LIST type runs the commands one after another.

Background (`echo bg &`). Both shells run the command in the background (BACK type) and print `bg`. Only `bash` has job control, so it also prints the messages `[1] 8915` and `[1]+ Done echo bg`.

Globbing (`ls *`). `bash` replaces `*` with the file names and prints `README f`. xv6 has no globbing: `ls` receives a literal `*` and prints `ls: cannot open *`.

Variables (`echo $HOME`). `bash` replaces the variable with its value, `/home/vboxuser`. xv6 has no variables, so the text `$HOME` is passed unchanged.

Quotes (`echo "a b"`). `bash` removes the quotes and prints `a b`. In xv6 the quotes

are ordinary characters, so they stay in the arguments and the output is `"a b"`.

PATH (`mkdir d ; cd d ; ls ..`). In `bash`, `cd` works inside a list and `ls` is found through `PATH`, so the output is `README d f`. In xv6, `cd` works only when it is alone on the line. Inside a list it is run as a program and `exec` fails with `exec cd failed`, so the directory does not change and `ls ..` lists the root directory.

History and completion (arrow up, Tab, Ctrl+R). These work in `bash`, because it uses the readline library. The xv6 shell has no history and no completion, because it reads a plain line from the console.

3.3 The PATH test in xv6

After `cd d` given alone on a line, the command `ls` fails:

```
$ mkdir d
$ cd d
$ ls
exec ls failed
$ ../ls
.              1 26 32
..             1 1 1024
$ /ls
.              1 26 32
..             1 1 1024
```

The commands `../ls` and `/ls` work, and `../ls ..` lists the root directory, where all the programs (`ls`, `cat`, `echo`, `sleep` and others) are stored.

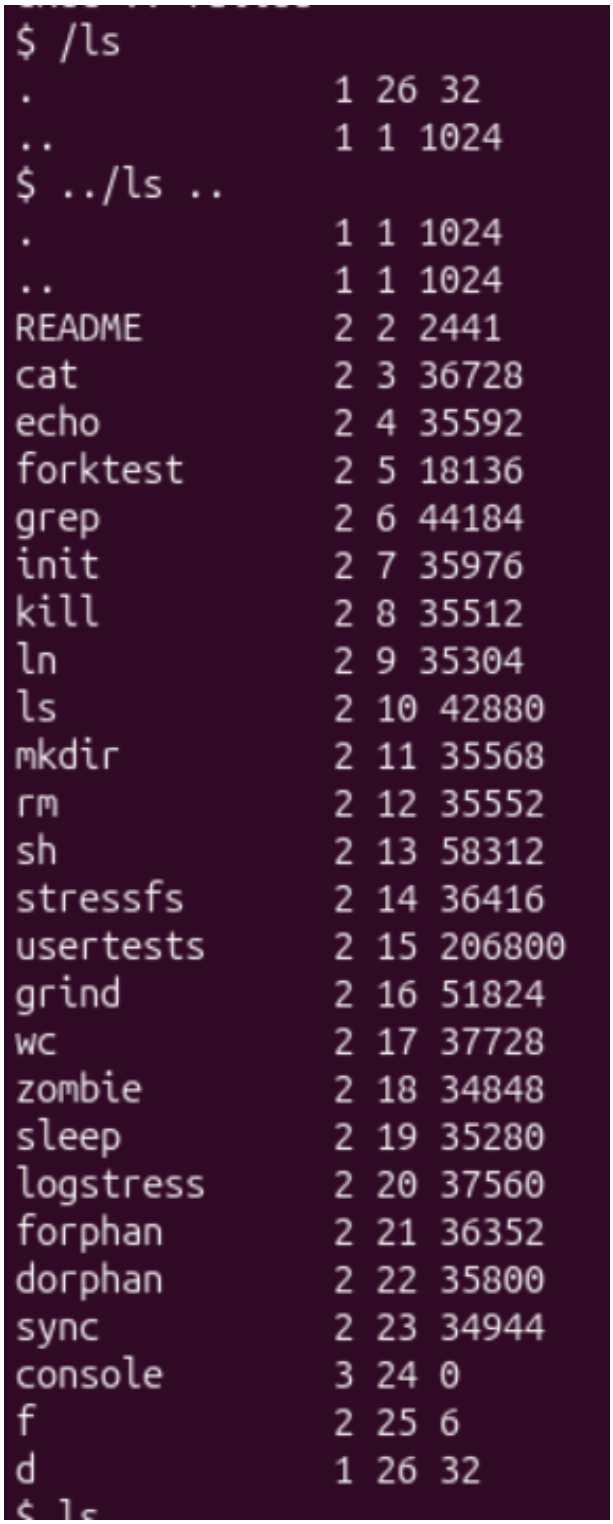


Figure 2.3 – ls failing after cd d, and the working alternative

3.4 Observe: answers

1. Features of both shells and features only in bash.

Both shells have: running programs, redirection (`<`, `>`), pipes (`|`), command sequences (`;`), background execution (`&`) and grouping with parentheses.

Only `bash` has: globbing (`*`), variables (`$HOME`), quote processing, job control messages,

the `PATH` search, command history and Tab completion.

2. Why does ls stop working after cd d?

The xv6 shell has no `PATH`. It calls `exec("ls")`, and a name without a slash is looked up relative to the current directory. After `cd d` the current directory is `d`, which has no file named `ls`. The xv6 programs live in the root directory `/`. Therefore `ls` can be run from inside `d` with the relative path `../ls` or the absolute path `/ls`.

3. Which missing feature would I miss most, and which is easiest to add?

The feature I would miss most in daily work is command history with Tab completion, because long commands would have to be typed again every time. Quote handling and globbing would also be missed. The easiest feature to add is a simple `PATH`: when `exec` fails, the shell can try the same name with the prefix `/`. Globbing would be harder, because the shell would have to read directories and match patterns.

4 PART 3. WHO EXPANDS *? A PROGRAM THAT PRINTS ITS ARGV

4.1 The program for xv6

The file `user/args.c` prints the number of arguments and each argument in square brackets:

```
#include "kernel/types.h"
#include "user/user.h"
int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}
```

The line `$U/_args\` was added to `UPROGS` in the `Makefile`, after `$U/_sleep\`.

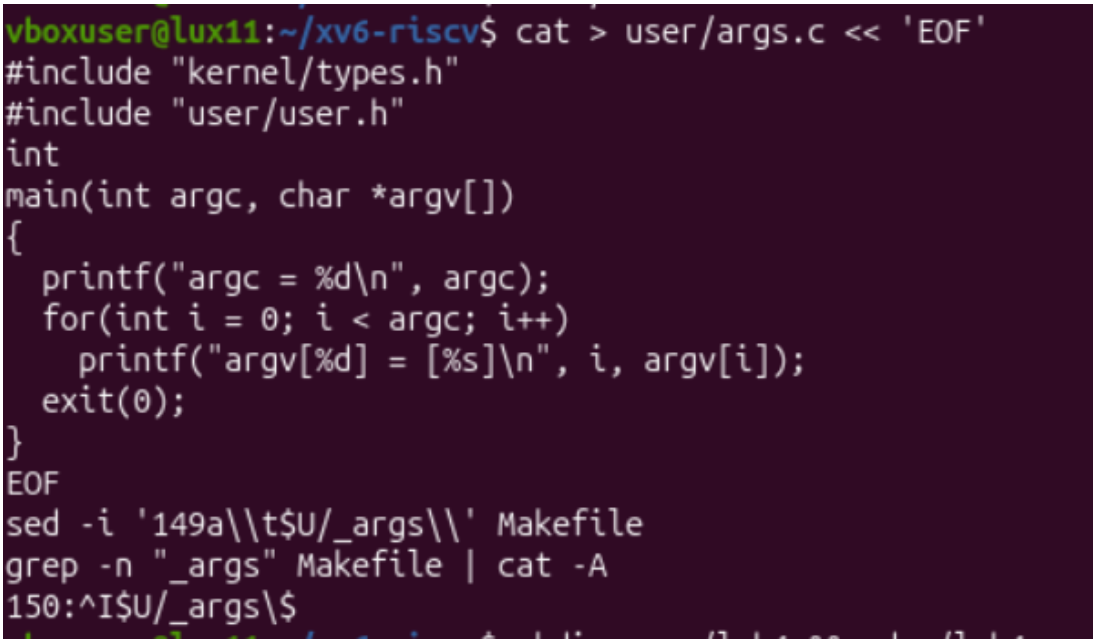


Figure 3.1 – args.c and the UPROGS line

4.2 The program for Linux

The file `args_linux.c` is the same program with `#include <stdio.h>` and `return 0;`:

```
#include <stdio.h>
int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
```

```
  return 0;
}
```

It was compiled with `gcc -o args args_linux.c` in a folder that contains the files `a.c`, `b.c` and `args_linux.c`.

4.3 Results in xv6

```
$ args one   two three
argc = 4
argv[0] = [args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
$ args *.c "x y"
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]
$ args $HOME
argc = 2
argv[0] = [args]
argv[1] = [$HOME]
$ args 'single quoted'
argc = 3
argv[0] = [args]
argv[1] = ['single]
argv[2] = [quoted']
```

4.4 Results in bash

```
$ ./args one   two three
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
$ ./args *.c "x y"
argc = 5
argv[0] = [./args]
argv[1] = [a.c]
argv[2] = [args_linux.c]
argv[3] = [b.c]
argv[4] = [x y]
$ ./args $HOME
argc = 2
argv[0] = [./args]
argv[1] = [/home/vboxuser]
$ ./args 'single quoted'
```

```
argc = 2
argv[0] = [./args]
argv[1] = [single quoted]
```

4.5 Observe: answers

1. Outputs for `args *.c "x y"` and the number of arguments.

In `bash` the program received 5 arguments: `./args`, `a.c`, `args_linux.c`, `b.c` and `x y`. In xv6 it received 4 arguments: `args`, `*.c`, `"x` and `y"`. The numbers are close, but the content is different.

2. Who turns `*.c` into file names and removes the quotes?

The shell does it. In `bash`, the shell replaces `*.c` with the matching file names and removes the quotes, and only then calls `exec` with the final list. The program does not do any of this: it only prints what it receives. The kernel does not do it either: it only copies the `argv` list into the new process. The xv6 shell only splits the line at spaces, so `*.c` stays literal and `"x y"` is split into two arguments with the quotes kept.

3. What happened to the extra spaces in `one two three`?

The shell uses spaces only as separators and discards them. The program receives the three arguments `one`, `two` and `three` without any spaces. This is the same in both shells.

5 PART 4. READING THE SHELL

5.1 Commands run

```
wc -l user/sh.c
grep -n "#define" user/sh.c
grep -n "case " user/sh.c
grep -n "cd" user/sh.c | head
```

The file `user/sh.c` has 499 lines. The `#define` lines show the kinds of commands:

```
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10
```

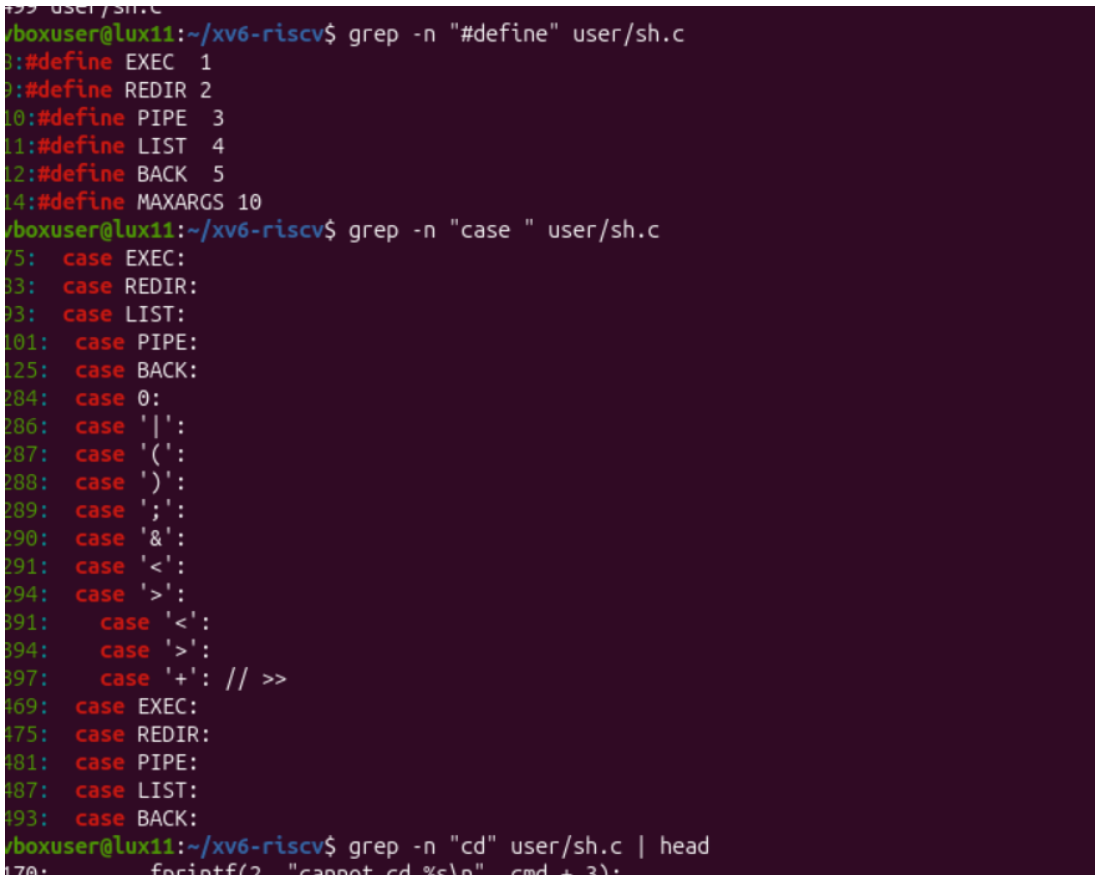


Figure 4.1 – The command kinds defined in sh.c

The function `runcmd` (lines 75–125) has one `case` for each kind: `EXEC`, `REDIR`, `LIST`, `PIPE` and `BACK`. The parser (lines 284–294) recognizes the special characters `| ( ) ; & < >`. The same five cases appear again in the function that builds the command structures (lines 469–493).

The handling of `cd` is in `main`, lines 160–176:

```
while (getcmd(buf, sizeof(buf)) >= 0) {
  char *cmd = buf;
  while (*cmd == ' ' || *cmd == '\t')
    cmd++;
  if (*cmd == '\n') // is a blank command
    continue;
  if (cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' ') {
    // Chdir must be called by the parent, not the child.
    cmd[strlen(cmd) - 1] = 0; // chop \n
    if (chdir(cmd + 3) < 0)
      fprintf(2, "cannot cd %s\n", cmd + 3);
  } else {
    if (fork1() == 0)
      runcmd(parsecmd(cmd));
    wait(0);
  }
}
```

5.2 Observe: answers

1. The five kinds of commands and the character that produces each.

Type	Character	Meaning
EXEC	plain words	a simple command, e.g. `ls -l`
REDIR	<, >, »	input/output redirection
PIPE	|	connects two commands with a pipe
LIST	;	commands run one after another
BACK	&	command run in the background

Parentheses only group commands; they are not a separate type.

2. Where is cd handled, and why is it not a separate program?

`cd` is handled in `main`, in the read loop (lines 160–176), before `fork1()`. If the line starts with `cd `, the shell calls `chdir()` in its own process. The current directory is a property of the process. A separate `cd` program would run in a child process, change only the directory of the child, and the shell would stay in the same directory after the child exits. Only the shell can change its own directory, so `cd` must be built into the shell. This also explains the result from Part 2: in `mkdir d ; cd d`, the line does not start with `cd `, so `cd` goes to `runcmd` and `exec("cd")` fails.

3. Code that handles * or $.

There is no code in `sh.c` that handles `*` or `$`. The special characters recognized by the parser are only `| ( ) ; & < >`. Quotes are not recognized either. This explains Part 2 and Part 3: `ls *` and `echo $HOME` pass the text unchanged, and quotes stay inside the arguments.

6 CONCLUSIONS

In this laboratory work the `sleep` program was fixed for the current xv6 by replacing the renamed system call `sleep()` with `pause()`, and it was added to `UPROGS`. It builds, runs and prints the usage message when the argument is missing.

The comparison between `bash` and the xv6 shell showed that the xv6 shell implements only the core of a shell: running programs, redirection, pipes, sequences, background execution and grouping. Globbing, variables, quote processing, `PATH`, job control, history and completion are additional features of `bash`. The missing `PATH` explains why `ls` stops working after `cd d`, and why `../ls` or `/ls` must be used.

The `args` program, run in both shells, proved that the shell expands `*`, replaces variables and removes quotes, and that neither the program nor the kernel does it. The kernel only passes the `argv` list to the new process.

Reading `user/sh.c` (499 lines) showed that the shell knows five kinds of commands: `EXEC`, `REDIR`, `PIPE`, `LIST` and `BACK`. It also showed that `cd` is a built-in command executed by the shell process itself, before `fork`, because the current directory belongs to the process.